

C-REV: INTELLIGENT CODE INSPECTION & PEER REVIEW STUDIO

A Multi-Tier Static Analysis, Dual-Pane Sandbox Execution & Differential Diagnostic Remediation System for Software Engineers

Project Name:	C-rev Studio (Code Reviewer)
System Type:	Full-Stack Web Engineering Tool
Frontend Stack:	React 18, Vite, Prism.js
Backend Stack:	Node.js, Express, Sandbox Engine
Author / Student:	Pradeep
Repository:	github.com/Pradeep8081/crev-code-reviewer
Documentation Date:	September 2026

1. Executive Summary & Problem Statement

In modern software engineering, peer code review and runtime error debugging are among the most critical yet time-consuming activities. Traditional tools typically present a binary outcome: either static linters that flag stylistic violations without runtime context, or cloud compilers that output raw, cryptic stack traces without actionable solutions.

C-rev solves this gap by building an integrated, developer-first engineering studio that combines:

- **Multi-File Workspace Management:** Supports uploading, editing, and batch-scanning multiple project files simultaneously.
- **Isolated Sandbox Execution Engine:** Spawns isolated child process environments to execute native Python and JavaScript/TypeScript routines safely, capturing execution duration, standard output, and runtime exceptions.
- **Differential Remediation & Word-Level Highlighting:** Automatically pinpoints the exact faulty sub-string expression (e.g. `acc.profile.tier`) and applies automated fixes (e.g. `acc.profile?.tier`) with sub-pixel zero cursor drift inside the live code editor canvas.
- **Pragmatic Peer Review:** Delivers concise health verdicts, Big-O time and space complexity ratings, and deep-dive architectural best practices tailored for working engineers.

Key Engineering Innovation

Unlike traditional linters that merely highlight entire lines, C-rev implements a custom Differential Token Isolator (`getChangedTokenInfo`) coupled with a Prism Private Unicode Delimiter Injection algorithm to highlight exact erroneous expressions in place without breaking syntax tokenization.

2. System Architecture & High-Level Design

The system follows a modern decoupled Client-Server architecture designed for high responsiveness, sub-second hot reloading, and secure process isolation.

1. CLIENT LAYER (FRONTEND)	2. API TRANSPORT LAYER	3. BACKEND RUNTIME ENGINE
Core: React 18 SPA + Vite Bundler Editor: react-simple-code-editor Syntax Lexer: Prism.js Tokenizer Markdown: React-Markdown + Rehype Styling: LeetCode Charcoal Dark Theme State: Workspace File Array, History LocalStorage, Active Diff Memo	Protocol: RESTful HTTP / JSON Client: Axios with 10MB payload limit Endpoints: • <code>POST /ai/run-code</code> (Execution & Crash Diagnosis) • <code>POST /ai/get-review</code> (AST Peer Review & Big-O) • <code>GET /</code> (Health & Engine Telemetry)	Runtime: Node.js v20+ / Express Sandbox: OS Child Process (<code>exec</code>) Timeout: 6,000ms safety watchdog Storage: Ephemeral OS temp files (<code>fs/promises</code>) Engine: Multi-model fallbacks (Gemini Flash Lite / 2.5 / 3.5)

2.1 Execution Data Flow Sequence

1. **User Input:** The developer writes or pastes code into the simple code editor canvas in the browser.
2. **Code Execution Trigger:** Clicking "Run Code" posts the snippet and detected language to `/ai/run-code`.
3. **Sandbox Isolation:** The Express backend writes the code to a cryptographically randomized temporary file in `os.tmpdir()` and executes it via a guarded child process with a 6-second watchdog timeout.
4. **Error Interception:** If an unhandled exception or non-zero exit code occurs, standard error (stderr) is captured and passed to the diagnostic remediation engine.
5. **Remediation Extraction:** The engine identifies the exact error line, faulty statement, corrected code, root cause, and suggested fix.
6. **Word-Level Highlighting:** The client executes `getChangedTokenInfo(oldLine, newLine)` to compute prefix/suffix bounds, injects Unicode boundary markers into Prism, and renders an in-place glowing green/red token highlight in the code editor.

3. Detailed Technology Stack & Rationale

Every library and framework in C-rev was deliberately chosen to meet specific performance and architectural requirements:

Component	Technology	Purpose & Why It Was Selected
Frontend Framework	React 18	Declarative component model, virtual DOM diffing, and fine-grained state management using hooks (<code>useMemo</code> , <code>useRef</code> , <code>useEffect</code>).
Build Tooling	Vite 8	Instant Hot Module Replacement (HMR) via native ES modules and optimized Rollup production bundling (520ms build times).
Code Editor Canvas	react-simple-code-editor	Lightweight, zero-dependency code editor that superimposes a transparent <code><textarea></code> over a synchronized <code><pre><code></code> syntax element. Avoids heavy Monaco bundle overhead.
Syntax Highlighting	Prism.js	High-speed regex grammar tokenizer supporting JavaScript, Python, TypeScript, C++, Java, and Go with modular theme stylesheets.
Markdown Parser	React-Markdown + Rehype-Highlight	Renders structured peer review reports with embedded syntax-highlighted code blocks securely without <code>dangerouslySetInnerHTML</code> vulnerabilities.
Icons & UI Assets	Lucide React	Crisp, tree-shakable SVG icon components representing developer tools (terminal, file check, shield, chevron, zap).
Backend Runtime	Node.js (v20+) & Express	Non-blocking, event-driven I/O model ideal for managing concurrent execution requests and file streams.
Execution Sandbox	child_process (exec)	Native OS process spawning with enforced 6-second timeout kill-signals and isolated temporary file lifecycle management.
Diagnostic Engine	@google/genai SDK	Low-latency inference models (Gemini Flash series) configured with strict JSON schemas and system instructions for AST and Big-O profiling.
Configuration	dotenv	Separates configuration from code; prevents accidental commitment of secret API keys to version control.

4. Core Algorithmic & Engineering Innovations

4.1 Differential Token Isolator: `getChangedTokenInfo`

Standard diff algorithms (like Myers Diff) operate at line or character granularity. When a line changes from:

```
console.log("User:", acc.username, "Tier:", acc.profile.tier);
```

to:

```
console.log("User:", acc.username, "Tier:", acc.profile?.tier);
```

a naive diff flags the entire 65-character line. C-rev's `getChangedTokenInfo` algorithm isolates the exact sub-expression:

```
function getChangedTokenInfo(oldStr, newStr) {
  if (!oldStr || !newStr || oldStr === newStr) return null;

  // 1. Compute common prefix
  let prefixLen = 0;
  while (prefixLen < oldStr.length && prefixLen < newStr.length && oldStr[prefixLen] === newStr[prefixLen])
    prefixLen++;
}

// 2. Compute common suffix
let oldSuffix = oldStr.length, newSuffix = newStr.length;
while (oldSuffix > prefixLen && newSuffix > prefixLen && oldStr[oldSuffix - 1] === newStr[newSuffix - 1])
  oldSuffix--;
  newSuffix--;
}

// 3. Expand outward to identifier boundaries ([a-zA-Z0-9_$?.\[\]])
function expandToken(str, start, end) {
  if (!str || str.length === 0) return '';
  let left = Math.max(0, Math.min(start, str.length - 1));
  let right = Math.max(0, Math.min(end, str.length));
  while (left > 0 && /[a-zA-Z0-9_$?.\[\]]/.test(str[left - 1])) left--;
  while (right < str.length && /[a-zA-Z0-9_$?.\[\]]/.test(str[right])) right++;
  return str.slice(left, right).trim();
}

return {
  oldWord: expandToken(oldStr, prefixLen, oldSuffix),
  fixedWord: expandToken(newStr, prefixLen, newSuffix)
};
}
```

4.2 Prism Private Unicode Delimiter Injection

Injecting HTML markup (like `<span class="highlight">`) directly into code before syntax highlighting corrupts Prism's regex lexer, causing HTML tags to be parsed as code tokens.

C-rev solves this by wrapping the target word with Unicode Private-Use Area characters:

- Start Marker: `\uFFFF0` | End Marker: `\uFFFF1`

Prism treats these characters as inert whitespace/text and highlights all internal keywords, strings, and operators normally. After Prism generates HTML, a single regex pass replaces the Unicode markers with zero-margin highlight spans:

```
const processed = line.replace(fixedWord, `\uFFF0${fixedWord}\uFFF1`);
const highlighted = prism.highlight(processed, grammar, lang);
return highlighted.replace(/\uFFF0([\s\S]*?)\uFFF1/g, '<span class="cr-code-word-fixed">$1</span>');
```

4.3 Zero-Drift Sub-Pixel Styling in `react-simple-code-editor`

Because `react-simple-code-editor` overlays a transparent `<textarea>` directly above the syntax-highlighted `<code>` element, any horizontal padding or margin on highlighted spans shifts the rendered characters and breaks cursor alignment.

C-rev enforces `padding: 0 !important; margin: 0 !important;` on all highlight spans, achieving 100% character alignment while displaying vibrant green background illumination and bottom border accents.

5. Professor & Viva Examination Preparation (Q&A)

These questions and model answers explain the exact technical workings of the project for oral defense:

Q1: Explain the high-level architecture of your project. What is the role of frontend and backend?

Technical Answer: C-rev uses a decoupled client-server architecture. The Frontend is built with React 18 and Vite for fast state management and dual-pane visual inspection. The Backend is an Express.js engine that provides two core services: an isolated OS child-process sandbox for runtime execution, and an AST profiling engine for automated peer review and Big-O analysis.

Say to Professor: "Sir, hamara frontend React 18 aur Vite par hai jo dual-pane editor manage karta hai, aur backend Node.js Express par hai jo code ko safely execute karta hai aur AST diagnostics nikalta hai."

Q2: How does the runtime execution sandbox work safely? Why doesn't it crash your server?

Technical Answer: Rather than executing untrusted user code in the main Node.js event loop (e.g. via dangerous `eval()`), the server writes the code to a randomized temporary file in `os.tmpdir()` and executes it as a separate OS child process via `exec()` with an enforced 6,000ms watchdog timeout. If code loops infinitely, the child process is killed without affecting the main server.

Say to Professor: "Sir, hum kabhi bhi eval() use nahi karte. Har code snippet ko OS ke temp directory me alag process ke roop me 6 second ke timeout ke saath run kiya jata hai. Agar code infinite loop me chala jaye, to sirf child process kill hota hai, server 100% safe rehta hai."

Q3: How do you achieve word-level highlighting without breaking syntax highlighting?

Technical Answer: We developed a two-stage algorithm. First, `getChangedTokenInfo` finds the common prefix and suffix between the old and new code lines and expands outward to identifier boundaries. Second, we wrap that word in non-breaking Unicode characters (`\uFFFF0` and `\uFFFF1`) before calling Prism. Prism highlights the code normally without corrupting HTML tags, and a post-processing regex substitutes the Unicode markers with styling spans with zero horizontal padding to prevent cursor misalignment.

Say to Professor: "Sir, agar hum Prism ke pehle HTML tag daalein to syntax coloring kharab ho jati hai. Isliye hum pehle invisible Unicode markers lagate hain, Prism syntax coloring karta hai, aur baad me hum markers ko zero-margin highlight span me convert kar dete hain."

Q4: What is the significance of the Dual Diff comparison card?

Technical Answer: Traditional linters show either the error or the fix in isolation. C-rev renders a synchronous side-by-side comparison displaying the exact faulty line with root-cause diagnostics on the red panel, and the corrected line with remediation explanation on the green panel. It also enables 1-click automated patching directly into the active editor state.

Say to Professor: "Sir, dual diff card me user ko ek taraf error wali line root cause ke saath red me dikhti hai, aur doosri taraf corrected code solution ke saath green me dikhta hai. User 1 click me code editor me fix apply kar sakta hai."

Q5: How is multi-file workspace and state persistence implemented?

Technical Answer: The workspace state is maintained as an array of file objects in React, each encapsulating file ID, name, code, language, active review markdown, run result, and applied fix metadata. We also maintain a 7-day auto-expiring local storage cache (`crev_code_history`) to preserve sessions across browser reloads without requiring an external database.

Say to Professor: "Sir, multi-file workspace React ke state array me maintain hota hai jisme har file ka code, language aur review cached rehta hai. Iske alawa 7-day expiration logic ke saath localStorage me session history save hoti hai."

6. How to Build This Project From Scratch (Step-by-Step Guide)

If another engineer or student wishes to replicate or extend this project, follow this structured roadmap:

Step 1: Backend Foundation & Sandbox (Node.js + Express)

1. Initialize Node project: `npm init -y` and configure `"type": "module"` in `package.json`.
2. Install runtime dependencies: `npm install express cors dotenv @google/genai`.
3. Create `server.js` and configure Express middleware (`cors()` , `express.json({ limit: '10mb' })`).
4. Implement child process execution using `exec()` with temporary files created via `fs/promises.writeFile()` and cleaned up in a `finally` block.
5. Implement diagnostic fallback: if child process returns error, pass error trace and code to Gemini Flash model with structured JSON response schema.

Step 2: Frontend Client Scaffold (React + Vite)

1. Initialize React Vite project: `npm create vite@latest Frontend -- --template react`.
2. Install dependencies: `npm install react-simple-code-editor prismjs lucide-react axios react-markdown rehype-highlight`.
3. Import Prism stylesheets (`prismjs/themes/prism-tomorrow.css`) and individual language grammars (javascript, python, cpp, java, etc.).
4. Create the LeetCode Charcoal design system in `App.css` with custom scrollbars, monospace typography, and split panes.

Step 3: Editor Integration & Differential Token Highlighter

1. Render the `<Editor>` component inside a canvas container synchronized with a custom line-number gutter.
2. Implement `getChangedTokenInfo(oldStr, newStr)` to isolate replaced expressions.
3. In the editor's `highlight` prop function, inject `\uFFFF0` and `\uFFFF1` around the identified word, run `prism.highlight()`, and substitute with `.cr-code-word-fixed`.

Step 4: Dual-Pane Studio & Tabbed Review Panel

1. Create the right-hand studio pane with 4 tabs: *Terminal*, *Peer Review*, *Diagnostics*, and *Architecture*.
2. Add a split-panel parser for review responses using the `---SPLIT_DETAIL---` delimiter to provide a concise verdict by default and an expandable deep-dive accordion on demand.
3. Add a global `keydown` listener for `Ctrl + Enter` to trigger audits seamlessly.

Step 5: Deployment & Git Version Control

1. Add a root `.gitignore` excluding `node_modules/`, `dist/`, and all `.env` files.
2. Publish the repository to GitHub: `git init && git branch -M main && git push -u origin main`.

7. Conclusion & Future Roadmap

C-rev demonstrates that modern software developer tools can achieve both high performance and deep automated remediation without compromising developer experience. By replacing generic conversational interfaces with a purpose-built, dual-pane studio layout, programmers obtain instant feedback, sub-pixel token highlighting, and verifiable runtime safety.

Future Enhancements:

- **Dockerized MicroVM Sandboxes:** Transition from OS child processes to containerized WebAssembly / Firecracker microVMs for cloud-scale multi-tenant isolation.
- **Git Pull Request Webhook Bot:** Automatically post dual diff remediation suggestions directly on GitHub / GitLab pull requests.
- **Custom Team Linting Rules:** Support custom organizational AST rules (e.g. ESLint / Pylint config ingestion).